

Code-level interfaces of External Encryptor + DLL interfaces/contracts

[Background](#)

[Encryptor/Decryptor Instantiation model](#)

[The EncryptorInterface and DecryptorInterface](#)

[EncryptorInterface](#)

[DecryptorInterface](#)

[Sequence diagrams](#)

[Data Batch Protection Agent](#)

[DBPA Shared Library: providing the Builder method](#)

[DBPA Shared library: instantiating DataBatchProtectionAgent](#)

[Agent Behavior contract and enforcement](#)

[The DBPAExecutor](#)

[DataBatchProtectionAgent interface](#)

[Class definition](#)

[Result container classes](#)

[Behavioral contract](#)

[Instantiating the External DBPA Encryptor](#)

[Instantiating ExternalDBPAEncryptorAdapter](#)

[Implementations of Data Batch Protection Agent](#)

[LocalDataBatchProtectionAgent](#)

[RemoteDataBatchProtectionAgent](#)

[Pros and cons of a remote agent implementation](#)

[Client side implementation](#)

[Server side implementation](#)

[Frequently Asked Questions](#)

[Appendix 1: source for Sequence Diagram](#)

[Appendix 2: Regarding shared libraries](#)

[Appendix 2.1: Existing dynamic library loading in Apache Arrow](#)

[Appendix 2.2: Arrow's Preference on Dynamic vs. Static linking](#)

[Appendix 2.3: Versioning of libraries](#)

Background

Today, Apache Arrow PME only has support for two internally-defined encryptors (AES-GCM and AES-GCM-CTR). We are extending PME's support so that externally-defined encryptors can be used.

The mental model is that external encryptors/decryptors are to be independent from the Arrow codebase. That means that external encryptors/decryptors can exist and operate independently of Arrow, and do not need to understand Arrow concepts/semantics in order to operate.

Following this model, we are defining the Data Batch Protection Agent (DBPA) - a new interface and entity for performing encryption/decryption functions. We will enable Arrow's integration and interoperability with DBPA implementations. Arrow will be a consumer of DBPA implementations, i.e. while Arrow will be made aware of DBPAs, DBPAs will exist outside/independent of Arrow.

Encryptor/Decryptor Instantiation model

In PME, encryptors are column-specific. There is an encryptor instance created for each column that requires encryption. This occurs in [InternalFileEncryptor::GetColumnEncryptor\(...\)](#) for column encryption (and in a few other, similar places in the same file for footer/metadata encryption).

The EncryptorInterface and DecryptorInterface

As part of our extensions, we defined two new interfaces which abstract the behavior of encryptors/decryptors, seen below:

EncryptorInterface

```
C/C++
class PARQUET_EXPORT EncryptorInterface {
public:
    /// Signal whether the encryptor can calculate a valid ciphertext length
    before
    /// performing encryption or not. If false, a proper sized buffer cannot be
    /// allocated before calling the Encrypt method, and Arrow must use this
    /// encryptor's EncryptWithManagedBuffer method instead of Encrypt.
    virtual bool CanCalculateCiphertextLength() const = 0;

    /// Calculate the size of the ciphertext for a given plaintext length.
    virtual int32_t CiphertextLength(int64_t plaintext_len) const = 0;

    /// Encrypt the plaintext and leave the results in the ciphertext buffer.
    /// Most implementations will require the key and aad to be provided, but it
    is
```

```

    /// up to each encryptor whether to use them or not.
    virtual int32_t Encrypt(
        ::arrow::util::span<const uint8_t> plaintext,
        ::arrow::util::span<const uint8_t> key, ::arrow::util::span<const
uint8_t> aad,
        ::arrow::util::span<uint8_t> ciphertext,
        std::unique_ptr<EncodingProperties> encoding_properties = nullptr) = 0;

    /// Encrypt the plaintext and leave the results in the ciphertext buffer.
    /// The buffer will be resized to the appropriate size by the encryptor
    during
    /// encryption. This method is used when the encryptor cannot calculate the
    /// ciphertext length before encryption.
    virtual int32_t EncryptWithManagedBuffer(
        ::arrow::util::span<const uint8_t> plaintext, ::arrow::ResizableBuffer*
ciphertext,
        std::unique_ptr<EncodingProperties> encoding_properties = nullptr) = 0;

    /// After the column_writer writes a dictionary or a data page, this method
    will be
    /// called so that each encryptor can provide any encryptor-specific column
    /// metadata that should be stored in the Parquet file. The keys and values
    are
    /// added to the column metadata, any conflicting key and value pairs are
    /// overwritten. There is no need to clear the metadata after the call.
    virtual std::shared_ptr<KeyValueMetadata> GetKeyValueMetadata(int8_t
module_type) {
        return nullptr;
    }

    /// Encrypt footer metadata for signature verification purposes only.
    /// This method is used specifically for footer signature verification in
    encrypted
    /// Parquet files with plaintext footers. It encrypts the footer metadata
    using
    /// the provided key, AAD, and nonce to generate an authentication tag that
    can
    /// be compared against a stored signature.
    virtual int32_t SignedFooterEncrypt(::arrow::util::span<const uint8_t>
footer,
        ::arrow::util::span<const uint8_t> key,
        ::arrow::util::span<const uint8_t> aad,
        ::arrow::util::span<const uint8_t> nonce,

```

```

encrypted_footer) = 0;
};

```

DecryptorInterface

```

C/C++
class DecryptorInterface {
public:
virtual ~DecryptorInterface() = default;

    /// Signal whether the decryptor can calculate a valid plaintext or
    ciphertext
    /// length before performing decryption or not. If false, a proper sized
    buffer
    /// cannot be allocated before calling the Decrypt method, and Arrow must use
    /// this decryptor's DecryptWithManagedBuffer method instead of Decrypt.
    virtual bool CanCalculateLengths() const = 0;

    /// Calculate the size of the plaintext for a given ciphertext length.
    virtual int32_t PlaintextLength(int32_t ciphertext_len) const = 0;

    /// Calculate the size of the ciphertext for a given plaintext length.
    virtual int32_t CiphertextLength(int32_t plaintext_len) const = 0;

    /// Decrypt the ciphertext and leave the results in the plaintext buffer.
    /// Most implementations will require the key and aad to be provided, but it
    is up to
    /// each decryptor whether to use them or not.
    virtual int32_t Decrypt(
        ::arrow::util::span<const uint8_t> ciphertext,
        ::arrow::util::span<const uint8_t> key, ::arrow::util::span<const
uint8_t> aad,
        ::arrow::util::span<uint8_t> plaintext,
        std::unique_ptr<EncodingProperties> encoding_properties = nullptr) = 0;

    /// Decrypt the ciphertext and leave the results in the plaintext buffer.

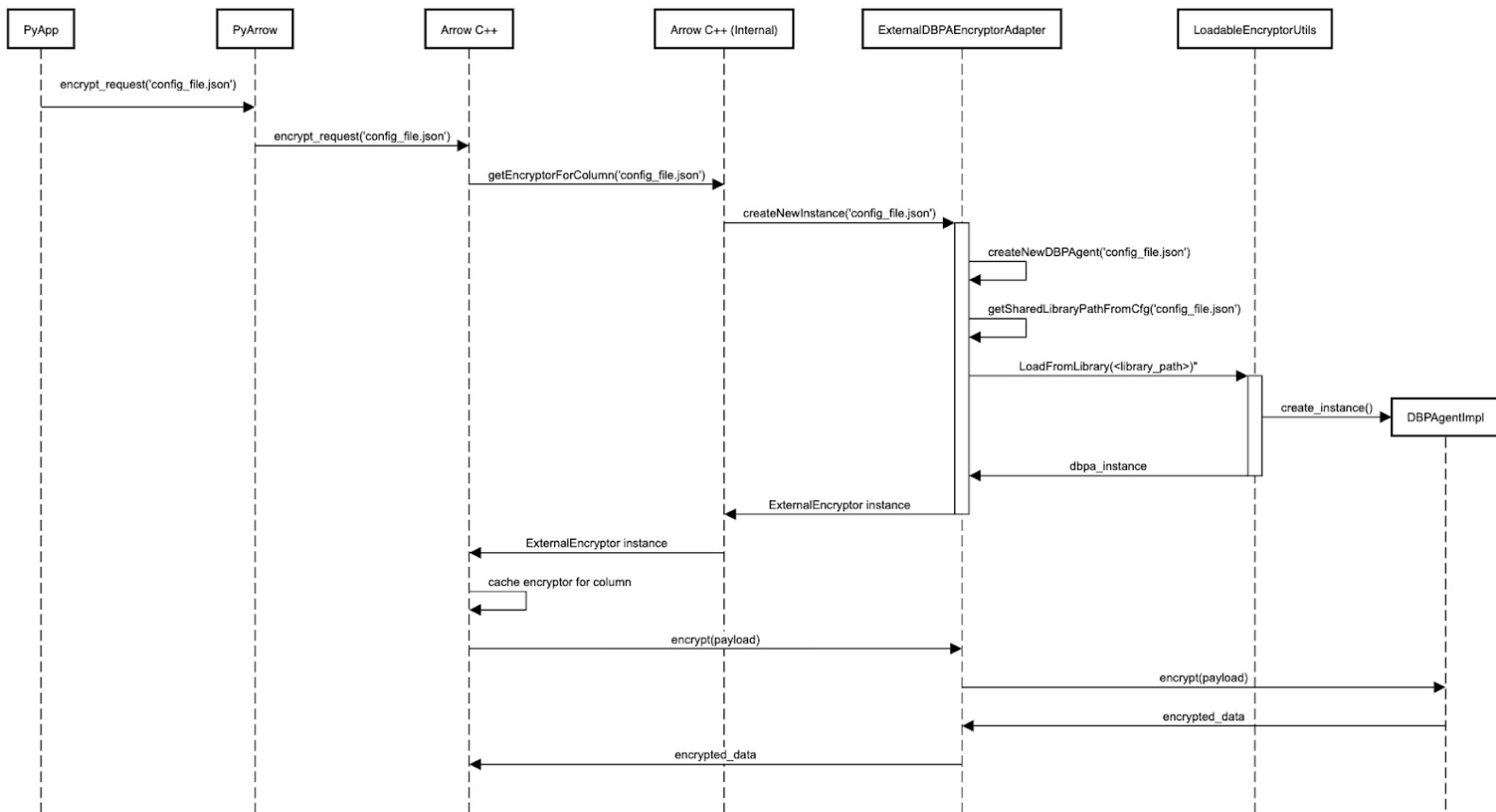
```

```

    /// The buffer will be resized to the correct size during decryption. This
    method
    /// is used when the decryptor cannot calculate the plaintext length before
    decryption.
    virtual int32_t DecryptWithManagedBuffer(
        ::arrow::util::span<const uint8_t> ciphertext, ::arrow::ResizableBuffer*
        plaintext,
        std::unique_ptr<EncodingProperties> encoding_properties = nullptr) = 0;
};

```

Sequence diagrams



Data Batch Protection Agent

The Data Batch Protection Agent (DBPA) is an external entity (i.e. independent of Apache Arrow) capable of performing encryption and decryption operations. `DataBatchProtectionAgent` is the interface

used by Arrow to invoke functionality by a DBPA. Implementations of DBPA are available to Arrow via shared libraries (e.g. *.so files).

A thorough description of `DataBatchProtectionAgent` can be found in this section:

[Code-level External Encryptor + DLL interfaces/contracts](#) .

DBPA Shared Library: providing the Builder method

Implementations of `DataBatchProtectionAgent` are loaded from a shared library (e.g. *.so file).

The shared library functions as a builder of `DataBatchProtectionAgent` objects. To achieve this, the library exposes a method called `create_new_instance()`. This method is exposed as a non-mangled symbol in the shared library in order to allow dependencies of the library to easily locate the method.

The `create_new_instance()` method must:

- Have a return type of `DataBatchProtectionAgent*`.
- Return a new instance of `DataBatchProtectionAgent` upon each invocation (i.e. singleton instances must not be returned)

The following code shows an example of both (a) the `create_new_instance()` method building an instance of `DataBatchProtectionAgent`, called `ExampleDBPAImpl()`, and (b) how to expose the non-mangled symbol (method name) in the symbol table of the shared library:

```
C/C++
extern "C" {
    DataBatchProtectionAgent* create_new_instance() {
        return new ExampleDBPAImpl();
    }
}
```

DBPA Shared library: instantiating DataBatchProtectionAgent

We created `LoadableEncryptorUtils`, a utils class which facilitates and abstracts the loading of the shared library, and the creation of `DataBatchProtectionAgent` instances

```
C/C++
class LoadableEncryptorUtils {
    static unique_ptr<DataBatchProtectionAgent> LoadFromLibrary(const string*
library_path);
}
```

```
};
```

The `LoadFromLibrary` method opens a handle to the shared library. Given that the library is the place where all the code for the `DataBatchProtectionAgent` exists, the library cannot be unloaded (i.e. its handle cannot be closed) until the `DataBatchProtectionAgent` instance is destroyed.

For this reason, we defined a new class `DBPALibraryWrapper` which is used as a decorator, therefore it has the same interface as `DataBatchProtectionAgent`. `DBPALibraryWrapper` contains (a) the reference of the instance of `DataBatchProtectionAgent` created by the shared library and (b) a reference to the shared library handle. All method calls to `DBPALibraryWrapper` are forwarded to the underlying `DataBatchProtectionAgent` instance. Whenever the destructor of `DBPALibraryWrapper` is invoked, it (a) invokes the destructor in the underlying `DataBatchProtectionAgent` instance (i.e. the *actual* implementation), and then (b) closes the shared library based on the reference to its handle.

In summary, the `LoadFromLibrary` method

1. Loads the shared library (provided by the `agent_library_path` parameter, e.g. [libdbpsLocalAgent.so](#)).
2. Maintains a reference to the shared library handle. Let's call it `library_handle`.
3. Invokes the `create_new_instance()` method provided by the library. This generates a new instance of `DataBatchProtectionAgent`. Let's call it `internal_instance`.
4. Creates a new instance (called `wrapped_instance`) of `DBPALibraryWrapper` and passes both `library_handle` and `internal_instance`.
5. Returns (the pointer to) `wrapped_instance`.

Agent Behavior contract and enforcement

Before our new extension, which allows for the incorporation of external encryptors, Arrow PME only contained an implementation of AES encryption (with two sub-flavors). The limited set of functions allowed for easier testing and controlling of behavior of said functions (i.e. latency requirements, reporting error conditions, etc).

With the PME extension, external encryptors are allowed to be loaded into Arrow. The nature of these external encryptors could be varied: while some implementations may simply implement a new local encryption algorithm, other implementations may make network calls to fulfil the encryption functionality. This added complexity brings additional risk to Arrow PME at runtime.

For this reason, we defined a set of rules/behavior/contract expectations that external encryptors must abide by. In order to enforce this contract, we defined a new class, `DataBatchProtectionAgentExecutor`, an Arrow-side component which is in charge of protecting

Arrow PME (and its calling application) from intentionally or un-intentionally misbehaving implementations of `DataBatchProtectionAgent`.

The DBPAExecutor

Implementations of the `DBPAExecutor` (aka the executor) are used via the decorator pattern. They share the same interface as the `DataBatchProtectionAgent` (DBPA). The executor wraps an actual instance of DBPA, and (1) forwards the encrypt/decrypt calls to the underlying DBPA, while also ensuring that the behavior contract is enforced.

The Executor ensures that these operations from the underlying agent complete within a reasonable time: `init()`, `encrypt()`, `decrypt()`. All exceptions raised in the underlying agent are propagated up to the caller (i.e. the application), and timeout exceptions are also thrown so that the caller (application) can catch them.

DataBatchProtectionAgent interface

The `DataBatchProtectionAgentInterface` virtual class (also called “DBPAInterface”) is the external interface of the DBPS framework. Arrow's `ExternalDBPAEncryptorAdapter` creates instances of `DataBatchProtectionAgentInterface` (more details [here](#)). The concrete implementations of DBPAInterface are transparent to Arrow.

This section describes the `DataBatchProtectionAgentInterface` and related classes independently of how these are built from the Arrow adapters (already covered in previous sections).

From the system architecture, DBPAInterface is the first level outside of Arrow. The DBPA classes and interfaces live in a separate code repository, called [DataBatchProtectionService](#). Arrow adds an external [dependency to the DBPS repository](#) to have the common headers and definitions (`${dbps_agent_SOURCE_DIR}/src/common`) available at compile time.

Class definition

```
C/C++
class DBPS_EXPORT DataBatchProtectionAgentInterface {
public:
    DataBatchProtectionAgentInterface() = default;

    virtual void init(...) {}

    /*
```



```

    * Encrypts the provided plaintext data using the configured encryption
parameters.
    *
    * @param plaintext Binary data to be encrypted, provided as a span of bytes
    * @param encoding_attributes A map of string key-values.
    * Each encoding type requires additional attributes to be properly
decoded.
    * @return A unique pointer to an EncryptionResult containing the encrypted
data and operation status
    */
    virtual std::unique_ptr<EncryptionResult> Encrypt(
        span<const uint8_t> plaintext,
        std::map<std::string, std::string> encoding_attributes) = 0;

    virtual std::unique_ptr<DecryptionResult> Decrypt(
        span<const uint8_t> ciphertext,
        std::map<std::string, std::string> encoding_attributes) = 0;

    /* Returns the encryption metadata provided during the class init() call.
    * The encryption metadata is a map of string key-value pairs and is
defined only for Decrypt usage.
    * This metadata map is the one returned during the Encrypt call.
    */
    virtual const std::optional<std::map<std::string, std::string>>
EncryptionMetadata() const {
        return column_encryption_metadata_;
    }

    virtual ~DataBatchProtectionAgentInterface() = default;

protected:
    std::string column_name_;
    std::map<std::string, std::string> configuration_map_;
    std::string app_context_; // includes user_id
    std::optional<std::map<std::string, std::string>>
column_encryption_metadata_;

    std::string column_key_id_;
    Type::type datatype_;
    std::optional<int> datatype_length_;
    CompressionCodec::type compression_type_;
};
}

```

Relevant attributes and parameters

- `configuration_map_:`
 - key-value pairs of strings used by the specific implementation for configuring the agent.
 - Examples of values are server url, connection pool attributes, or threading specific values.
 - These are meaningless at the Arrow and DBPAinterface level, and are only interpreted by the specific DBPA implementations.
- `app_context_:`
 - Json formatted string with additional information context at the application level.
 - Used by context-aware encryption agents to make low-level customizations. These customizations are specific to the implementation and may include specific ACL rules, encryption specifics by geolocation, or fine-tuned based on database schemas.
 - This is a specific contract between the Arrow application layer and the encryptor, passed as-is by the Arrow libraries.
- `column_encryption_metadata_:`
 - key-value pairs of strings to expose encryption-specific parameters generated during encryption or decryption.
 - It is the per-column encryption contract. The metadata produced at encrypt time is passed back at decrypt time so the agent knows how to handle the ciphertext.
 - The encryption operation creates: `dbps_agent_version` (the version of DBPS that did the encryption) and `encryption_mode` applied (`per_value` or `per_block`).
 - The upstream Arrow caller writes the values as metadata in the Parquet file and sends them back during the Decryption operation of the column.
- `datatype_and datatype_length_:`
 - Enum to indicate the type of the elements in the plaintext or ciphertext buffer (`BOOLEAN`, `INT32`, `INT64`, `INT96`, `FLOAT`, `DOUBLE`, `BYTE_ARRAY`, `FIXED_LEN_BYTE_ARRAY`)
 - From the datatype, the size of the elements and number of elements can be inferred. This is used to parse and make per-value encryption, either as raw data or as typed values.
 - This typing information can be used by a DBPA implementation with type-specific encryptor library functions.
- `compression_type_:`
 - Enum to indicate the compression algorithm or library applied to the payload.
- **Encrypt/Decrypt** `encoding_attributes:`
 - Parquet page specific encoding key-value pairs. Parquet pages have 2 parts: level bytes and value bytes. Both parts need to be processed, although the per-value operation only applies to the value bytes portion of the payload.
 - During encryption, these parameters are required for the agent to adequately parse the plaintext buffer.

- During decryption, these parameters are needed to transform the ciphertext and generate back a correctly formatted Parquet page in the same form as it was received on the encrypt call.
- The agents require a Parquet specific library to parse the payloads.

Result container classes

C/C++

```
class DBPS_EXPORT EncryptionResult {
public:
    virtual span<const uint8_t> ciphertext() const = 0;

    // Allows a larger backing buffer than the exact ciphertext size.
    // Library users must check size() to ensure the actual size of the
    returned payload.
    virtual std::size_t size() const = 0;

    // Success flag; false indicates an error.
    virtual bool success() const = 0;

    // Encryption metadata (valid when success() == true)
    // Map of string key-value pairs parameters produced during encryption that
    are needed for decryption,
    // for example, the encryption_algorithm_version used.
    virtual const std::optional<std::map<std::string, std::string>>
    encryption_metadata() const = 0;

    // Error details (valid when success() == false).
    virtual const std::string& error_message() const = 0;
    virtual const std::map<std::string, std::string>& error_fields() const = 0;

    virtual ~EncryptionResult() = default;
};
```

C/C++

```
class DBPS_EXPORT DecryptionResult {
public:
    virtual span<const uint8_t> plaintext() const = 0;

    // Allows a larger backing buffer than the exact plaintext size.
    // Library users must check size() to ensure the actual size of the
    returned payload.
```

```

virtual std::size_t size() const = 0;

// Success flag; false indicates an error.
virtual bool success() const = 0;

// Error details (valid when success() == false).
virtual const std::string& error_message() const = 0;
virtual const std::map<std::string, std::string>& error_fields() const = 0;

virtual ~DecryptionResult() = default;
};

```

Behavioral contract

Implementations of the `DataBatchProtectionAgentInterface` class must comply with the following:

- While a handle to `EncryptionResult/DecryptionResult` object exists, `ciphertext()/plaintext()` is guaranteed to be retained and return the valid text. The ciphertext/plaintext is fixed and not consumed if read.
- Read operations are not destructive. Multiple calls return the same data.
- Destructor must dispose of internal memory (either by delegation or cleanup)
- Library users must check `size()` to ensure the actual size of the returned payload.
- Exceptions on `init(): init()` may throw `DBPSEException` for initialization errors (e.g., invalid parameters, server connection failures)
- Exceptions on `Encrypt()` and `Decrypt()`: Encrypt/Decrypt do not throw exceptions. Errors reported via `success()` flag and error methods.

Instantiating the External DBPA Encryptor

As mentioned, the mental model is for DPBA implementations to be consumed by Arrow. This means that external encryptors/decryptors do not need to understand Arrow concepts/semantics/entities in order to operate. For this reason, we require an adapter component which helps the DBPA meet Arrow's specifications (i.e. interfaces). These adapters exist in the Arrow side, and are called `ExternalDBPAEncryptorAdapter` and `ExternalDBPADecryptorAdapter`. These adapters implement the `EncryptorInterface` and `DecryptorInterface` respectively (described [above](#)). The adapters perform these high level functions: (a) understanding the behavior of the external encryptor/decryptor, (b) making use of the external encryptor/decryptor functionality in the DBPA, (c) protecting Arrow from a potentially mis-behaving external DBPA implementation.

To achieve this, the adapters (1) load the external defined DBPA implementations from a shared library (e.g. *.so file), (2) invoke the respective encrypt/decrypt functions inside the DBPA implementation, and (3) make use of the `DBPAExecutor` to protect Arrow from devious behavior in the DBPA. As per PME

existing behavior, encryptor instances are not shared between columns - this means that Arrow will create a separate instance of an encryptor (both internal and external) for each column that requires encryption.

Instantiating ExternalDBPAEncryptorAdapter

When creating a new instance of `ExternalDBPAEncryptorAdapter`, the adapter:

1. Reads `agent_library_path` from `configuration_properties`, the file name of the shared library which contains the DBPA agent.
2. Invokes `LoadableEncryptorUtils::LoadFromLibrary(library_path)`, assigns the return to `agent_instance` (as mentioned [earlier](#), this will be a DBPA instance wrapped in a `DBPALibraryWrapper` decorator).
3. Instantiates `dbpa_executor`, a `DBPAExecutor` decorator which will wrap `agent_instance`.
4. Invokes `dbpa_executor->init(...)`.
5. Maintains an internal reference to `dbpa_executor`, which will be used later for performing the encrypt/decrypt operations.

Implementations of Data Batch Protection Agent

There are 2 implementations of the `DataBatchProtectionAgentInterface` class. They live in the [DataBatchProtectionService](#) codebase. Arrow interacts solely with `DataBatchProtectionAgentInterface`, so the implementation details remain hidden. The implementations follow the behavioral contract described above.

LocalDataBatchProtectionAgent

[LocalDataBatchProtectionAgent](#) is an in-process implementation of `DataBatchProtectionAgentInterface`. It is a lightweight alternative, suitable for uses where the protection logic runs in the same process. Being in-process means the computation of encryption and decryption is executed locally resulting in a larger CPU and memory footprint, but with no immediate network load.

The actual encrypt and decrypt functions are executed by the [EncryptionSequencer](#) library.

RemoteDataBatchProtectionAgent

[RemoteDataBatchProtectionAgent](#) is a network-based implementation. Instead of running encryption/decryption logic in-process, it delegates the operations to an external DBPS server communicating via a [RESTful API](#). From the Arrow perspective, the remote agent behaves identical to the local implementation. The fact that operations run remotely is transparent to Arrow.

The off-process delegation means lower local CPU and memory resources with a higher latency due to off-process computation.

Pros and cons of a remote agent implementation

Having an off-process DBPS server serves multiple purposes:

- Security and key management: Encryption keys and credential validation can be hidden from the Arrow client. The server manages them separately or these are delegated to other services.
- Access control: The server can enforce various levels of access to data (per-user, per-column, or per-key-id). Arrow applications cannot be bypassed since the checks are done remotely.
- Downstream dependencies: The encryption libraries themselves and other library integrations are centralized on the DBPS server, making it simpler to administer centrally.
- Computational capacity: Delegation of the encrypt/decrypt computation lessens the computational resources of the local setup and permits fine-tuning of dedicated DBPS service.
- Separation of layers and behavior consistency: A remote server results in a cleaner separation of layers and abstracts logic better. A centralized encryption server also results in more consistent behavior, simplifies the deployment of new versions, and makes the overall behavior more easily observable (usage, errors).

Trade-off points:

- Network latency vs higher throughput: Off-process (i.e. network-based) encryption means every function call spends additional time on RPC calls through the wire, compared to making a local function library call. However, for processing high-volume encryption requests as required in large Parquet files, a local setup can be limited. A remote/network based implementation which takes advantage of parallel calls and batching can offset the performance penalties of the RPC calls.

Client side implementation

The Arrow facing client-side library that implements the `DataBatchProtectionAgentInterface` is the [RemoteDataBatchProtectionAgent](#). Similar to the local version, an instance of the agent is initialized once per column. It establishes and manages an HTTP connection pool to the server, and handles the calls to encrypt/decrypt belonging to that column. All encrypt/decrypt operations happen on the server side.

The agent uses [HttpClient library](#) to manage the typical network level conditions: token session fetching, connection handling, retries, error handling, and non-transient errors. This is a separation of logical vs http requests to keep abstractions separate.

During initialization:

- The agent stores the parameters reused for every encrypt/decrypt request for the assigned column. This is common behavior for any agent implementation.
- The agent then loads the connection config file passed on the `configuration_map_` and extracts the server URL, pool settings and credentials. It then creates a shared pooled HTTP

client for the server URL. The connection pool is shared across agent instances pointing at the same server. An initial `/healthz` check is sent to the server.

- Then [the credentials are read](#) from the `configuration_map_`. Credential keys are prefixed with ``credentials.`` so they can be identified and collected in a generic way. Credentials are provided directly by the Arrow application and passed as-is from app to `RemoteDataBatchProtectionAgent`.
- Next a session JWT authentication token is prefetched and cached for use on subsequent calls using the credential values. This is when the credentials are sent to the server. The agent itself simply forwards the `configuration_map_.credentials` to the server on the `/token` endpoint call.

During encrypt and decrypt calls:

- The agent sends a single encrypt/decrypt request to the server for each call. It uses the [RESTful API](#) to send the HTTP requests.
- It builds the JSON request with the cached session JWT, the column parameters, the Parquet `encoding_attributes`, encodes the payload as base64, and adds a timestamp-based reference ID for remote tracking. It then borrows a connection from the pool and sends the HTTP request.
- It then parses the JSON response and validates fields against the request (compression for encrypt and datatype + compression for decrypt) to check for request-response consistency.
- If during the encrypt/decrypt call the session JWT expires, the token is refreshed via a `/token` endpoint call and the encrypt/decrypt request is retried.

Server side implementation

[The DBPS server](#) is a multithreaded HTTP server built on the Crow framework that receives encrypt and decrypt requests from the remote agents. It implements the server side of the [RESTful API](#).

It delegates the actual encrypt/decrypt function to the same [EncryptionSequencer](#) library used by the local agent. Functionally, it is a thin, stateless service. It holds no per-column or per-session state. All the context needed to process a request is contained in the request itself.

The server is configured at [startup with a credentials file](#) and a JWT secret key to [generate session JWTs](#) during authentication `/token` requests, and to [validate JWTs](#) during encrypt/decrypt endpoint calls.

Beyond that it requires no external dependencies to handle requests.

Frequently Asked Questions

Q. Given the model where there is one encryptor/decryptor instance per column, are there any concerns regarding the number of times that the shared library may be open?

This is not a concern. Operating systems (OS) are smart enough to only open a shared library once, even if multiple requests are made to keep it open. To ultimately close the shared library, the OS uses reference counting to keep track of the number of 'open' handles to the library. Once the count gets to zero, the library is closed and removed from memory.

Q. Can we have class-level members for classes defined within a shared library?

Yes, this is not a concern. We have tested (in linux) that classes loaded from a shared library can have class members (e.g. connection pools, singletons, etc). From a coding (i.e. class writing) perspective, there is no special handling needed - using class members as one would in a 'regular' application will have the same effect in classes loaded from shared libraries. The caveat here: this behavior is only guaranteed if the library is loaded from the same file (i.e. file handle) - separate shared library files (even if they are identical) are treated differently by the operating system and will be loaded in a separate memory space ([\[1\]](#), [\[2\]](#)).

Q. In Arrow, Encryptors and Decryptors are separate entities. Will they share an implementation of an agent for the same column?

No, external agents will never be shared amongst encryptor/decryptors, nor amongst columns.

The Arrow model separates the encryption and decryption concepts, and their usage is generally focused in one operation: either encryption (while writing out a file), or decryption (while reading a file). Based on that, we feel that it is safe from both a functional and performance point of view to maintain an agent instance for each encryptor/decryptor. We do not see any value in having separate `DataBatchProtectionAgent` instances for encryptors and decryptors - specially given that implementing a shared agent amongst encryptors/decryptors requires a non-trivial amount of work.

Appendix 1: source for Sequence Diagram

Created using <https://sequencediagram.org/>

```
participant PyApp
participant PyArrow
participant Arrow C++
participant Arrow C++ (Internal)
participant ExternalDBPAEncryptorAdapter
participant LoadableEncryptorUtils

PyApp->>PyArrow:encrypt_request('config_file.json')\n
PyArrow->>Arrow C++:encrypt_request('config_file.json')
```



```

Arrow C++->Arrow C++ (Internal): getEncryptorForColumn('config_file.json')

Arrow C++ (Internal)->ExternalDBPAEncryptorAdapter:
createNewInstance('config_file.json')
activate ExternalDBPAEncryptorAdapter
ExternalDBPAEncryptorAdapter->ExternalDBPAEncryptorAdapter:createNewDBPAgent('
config_file.json')

ExternalDBPAEncryptorAdapter -> ExternalDBPAEncryptorAdapter:
getSharedLibraryPathFromCfg('config_file.json')
ExternalDBPAEncryptorAdapter ->
LoadableEncryptorUtils:LoadFromLibrary(<library_path>) "
activate LoadableEncryptorUtils
LoadableEncryptorUtils->*DBPAgentImpl: create_instance()
LoadableEncryptorUtils -> ExternalDBPAEncryptorAdapter: dbpa_instance
deactivate LoadableEncryptorUtils
ExternalDBPAEncryptorAdapter->Arrow C++ (Internal): ExternalEncryptor instance
deactivate ExternalDBPAEncryptorAdapter
Arrow C++ (Internal)->Arrow C++: ExternalEncryptor instance
Arrow C++ -> Arrow C++: cache encryptor for column
Arrow C++->ExternalDBPAEncryptorAdapter: encrypt(payload)
ExternalDBPAEncryptorAdapter -> DBPAgentImpl:encrypt(payload)

ExternalDBPAEncryptorAdapter <- DBPAgentImpl:encrypted_data

Arrow C++<-ExternalDBPAEncryptorAdapter:encrypted_data

```

Appendix 2: Regarding shared libraries

Appendix 2.1: Existing dynamic library loading in Apache Arrow

A. Dynamic Library Loading Utilities

Arrow's codebase contains a cross-platform utility for loading dynamic libraries ([arrow/util/io_util.cc](https://github.com/apache/arrow/blob/master/util/io_util.cc)). While loading of dynamic libraries is platform-specific, the [LoadDynamicLibrary](https://github.com/apache/arrow/blob/master/util/io_util.cc) method abstracts away the complexity/differences for Windows and Unix-like platforms. This method receives the library name and/or path to a library as an input, and as an output it either returns a handle to the library if successfully loaded and open, or an error code (nullptr). To locate the specific library, the method relies on the specific Operating System mechanisms for loading libraries (which means either using the fully-qualified path of the library, or making use a set of pre-determined locations for libraries).

Additionally, Arrow also provides utilities for loading specific symbols (i.e. methods) once a library has been loaded. Also in [io_util.cc](#), the [GetSymbol](#) method abstracts the cross-platform complexities of the symbol resolution.

B. Usage of Dynamic Libraries

Currently, the main use Arrow makes of the `io_util.cc` mentioned above is to perform the loading of file system implementations. In particular, in [arrow/filesystem/filesystem.cc](#), Arrow seems to attempt to optionally load the file system registry via an external library.

Additionally, external library loading is used for the HDFS file system implementation shim ([arrow/io/hdfs_internal.cc](#))

C. Other uses of Dynamic Libraries

We identified another use of dynamic libraries, however this is not related to extending Arrow's functionality as a platform, but instead is used to implement specific internal features/optimizations. In particular it has to do with the use of *Gandiva*, a just-in-time expression compiler built on top of LLVM ([\[i\]](#), [\[ii\]](#), a code optimizer and generator), designed for improving the performance of vectorized SQL-like expressions on Arrow columnar data. Gandiva makes use of LLVM via dynamic library loading ([gandiva/engine.cc](#)).

Appendix 2.2: Arrow's Preference on Dynamic vs. Static linking

The Arrow documentation does not show clear guidance around whether dynamic vs static linking of dependencies and libraries is preferred. However, the [Arrow documentation](#) does mention that *"In most cases, it is recommended to use the Arrow shared libraries."*

By exploring the CMake (build system) configuration files ([cpp/CMakePresets.json](#)), we encounter that the static linking is turned off by default via the assignment of the build variable `ARROW_BUILD_STATIC` to `OFF`, which is used throughout the building and linking of both the Arrow core package and its dependent libraries.

Additionally, according to the CMake file for building Arrow's modules ([cmake_modules/DefineOptions.cmake](#)), all currently existing modules are built to be dynamically linked. This happens via the more general build variable `ARROW_DEPENDENCY_USE_SHARED`, which is set to `ON`. The value of said variable is used as the default for deciding whether to build individual modules as shared libraries or statically linked.

These points seem to indicate that Arrow has a preference for dynamically linking of libraries and modules.

Appendix 2.3: Versioning of libraries

For Unix-like OS's (linux, MacOS)

- Versioned Libraries are stored as the full filename, which represents the library version (e.g. [libfoo.so.2.3.4](#))

- The 'official' library to be used by applications is generally a sym-link representing the major version (e.g. `library.so.2` -> `library.so.2.3.4`).
- Applications tend to load library just for the name (`library.so`) or the name + major version (`library.so.2`)
- Versioned libraries are typically stored in standard directory hierarchies: `/usr/lib`, `/usr/local/lib`, or `/lib`. On modern 64-bit Linux systems, you often see `/usr/lib64` or architecture-specific paths like `/usr/lib/x86_64-linux-gnu`.

For Windows

- Typically, there is no global way to version libraries, and no sym links are used. If an app needs version 2, the file is generally called `libfoo_v2.dll`.
- Libraries are typically found in the application directory, system directories (e.g. `C:\Windows\System32`, `C:\Windows\SysWOW64`)
- Disambiguating library versions in Windows is agreed to be harder and more complex than in Unix-like OS's.